

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ «ЛЭТИ»
ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра информационных систем

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Организация ЭВМ и систем»
Тема: Исследование видеосистемы (графический режим)

Студенты гр. 4372

Климаш И.В.

Преподаватель

Подклетнов С. Г.

Санкт-Петербург

2025

Цель работы.

Изучение работы с видеосистемой в графическом режиме, вывод графика заданной функции с масштабированием и разметкой осей.

Задание на лабораторную работу.

1. Разработать программу для вывода на экран графика заданной функции (рисунок 1).

10	$\sin^3(x) + \cos^3(x)$	$\pi/2$	6π
----	-------------------------	---------	--------

Рисунок 1 – Заданная функция (вариантное задание)

2. Произвести разметку осей и проставить истинные значения точек.
3. Найти максимальное значение функции на заданном интервале и вывести в отдельное окно на экране.

Пример запуска программы

1. При запуске программы отображается график заданной функции согласно варианту (рисунок 3):

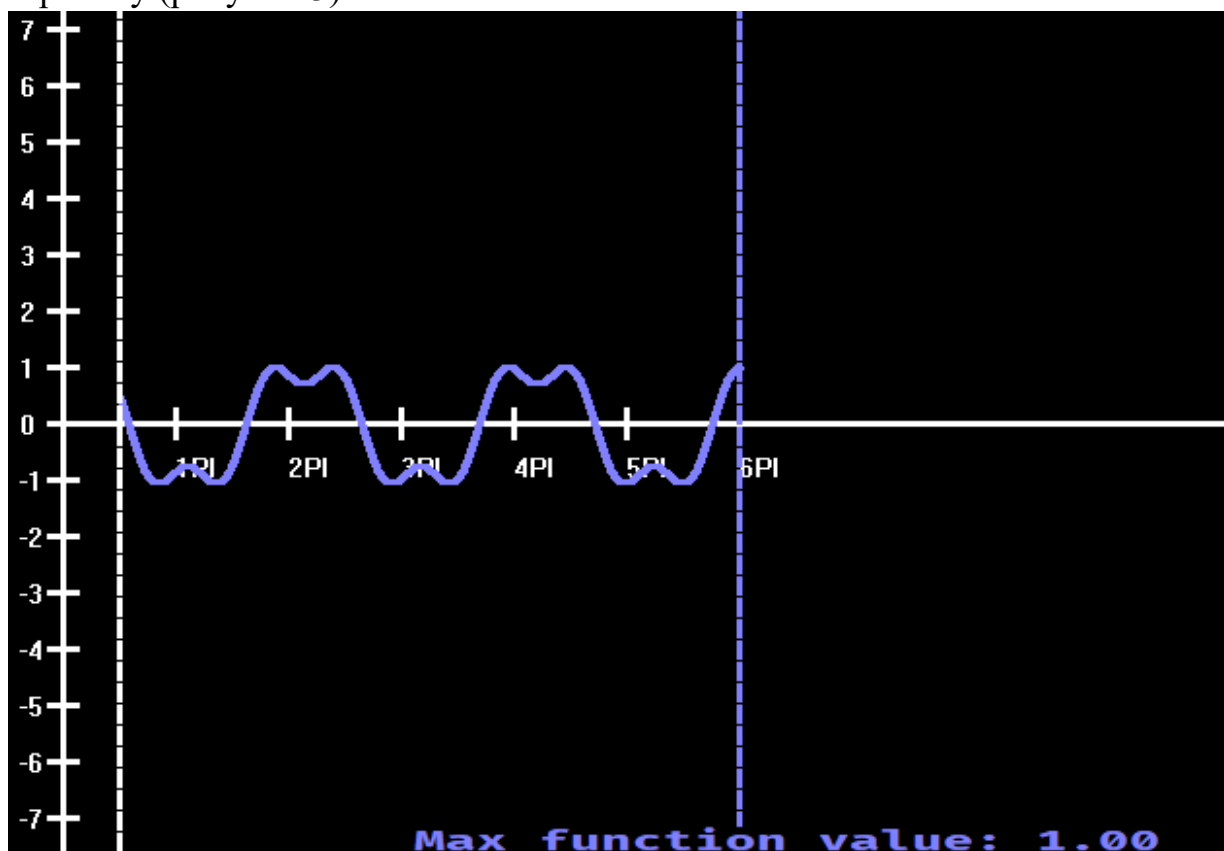


Рисунок 3 – Отображение графика функции

Как представляется цвет в видеосистемах?

Разность цветов и проблема черного.

Цвет — это не просто эстетическое понятие, а сложный результат взаимодействия физики, биологии и технологий. В видеосистемах представление цвета является фундаментальной задачей, от решения которой зависит качество и реалистичность изображения.

1. Основные цветовые модели в видеосистемах

1.1 Аддитивная модель RGB

Модель RGB (Red, Green, Blue) является фундаментальной для всех устройств, излучающих свет, таких как мониторы, телевизоры и проекторы. Ее принцип основан на аддитивном смешении (сложении) трех основных цветов. Комбинируя их в различных пропорциях, можно получить широкий спектр оттенков. Каждый пиксель на дисплее состоит из трех субпикселей — красного, зеленого и синего. При максимальной интенсивности всех трех компонентов получается белый цвет, а при их полном отсутствии — черный.

1.2 Субтрактивная модель CMYK

Для полноты картины стоит упомянуть и субтрактивную модель CMYK (Cyan, Magenta, Yellow, Key/Black), используемую в полиграфии. В отличие от аддитивной RGB, она основана на вычитании цветов: краски на бумаге поглощают определенные части спектра белого света, отражая лишь остаточный цвет. Хотя эта модель напрямую не используется в видеосистемах, понимание ее принципа помогает осознать разницу между излучаемым и отражаемым светом.

2. Разностные модели кодирования цвета: YUV/YCbCr

Чтобы преодолеть недостатки RGB, была разработана модель YUV (также известная как YCbCr). Это разностная модель, которая разделяет информацию о изображении на два отдельных компонента. Y (Luma) — компонент яркости. Это черно-белая, монохромная составляющая изображения, несущая информацию о деталях и контрасте. Человеческий глаз чрезвычайно чувствителен к изменениям яркости. U и V (Cb и Cr) — компоненты цветности (Chroma). Они содержат информацию о цвете, представляя собой разности между яркостью и синим (B-Y) и красным (R-Y) каналами соответственно. Глаз гораздо менее восприимчив к тонким изменениям цветности.

3. Проблема отображения черного на дисплеях.

ЖК-мониторы (LCD): В этих дисплеях используется постоянная подсветка. Для отображения черного жидкие кристаллы блокируют свет, но делают это неидеально. В результате черный цвет выглядит как темно-серый, что снижает контрастность изображения.

OLED-мониторы: Здесь проблема решена кардинально. Каждый пиксель является самостоятельным источником света и может быть полностью выключен для отображения идеального черного. Это обеспечивает бесконечную контрастность.

4. Природа фиолетового цвета.

Фиолетовый — неспектральный цвет. В физическом спектре он находится на краю видимого излучения с самой короткой длиной волны. Однако тот насыщенный фиолетовый, который мы часто видим на экранах, является результатом смешения синего и красного света. Наш мозг интерпретирует одновременное возбуждение "синих" и "красных" колбочек сетчатки как фиолетовый цвет. Это делает его особенно зависимым от точности передачи обеих компонент.

Заключение.

Представление цвета в видеосистемах — это сложный компромисс между точностью воспроизведения, эффективностью кодирования и особенностями человеческого восприятия. Модель YUV/YCbCr с субдискретизацией цветности стала отраслевым стандартом, позволившим значительно сократить объем данных при сохранении высокого визуального качества. Однако этот подход выявляет специфические challenges, связанные с воспроизведением определенных цветов.

Черный цвет остается пробным камнем для технологий отображения, демонстрируя разницу между идеалом (полное отсутствие света в OLED) и компромиссной реальностью (темно-серый в LCD). Фиолетовый цвет, будучи продуктом нашего восприятия и сложной комбинации сигналов, особенно уязвим к ошибкам кодирования, баланса белого и обработки изображения.

Рабочий код

```
#include <stdio.h>
#include <stdlib.h>
#include <graphics.h>
#include <math.h>
int main() {
    const double pi = 3.14159265358979323846;
    const double x_approach = 60 / pi;
    const double y_approach = 30;

    const int ten_num = 15;
    const float start_out = 0.5;
    const int end_out = 6;

    const int hatch_out = 58;
    const int number_out = 15;
    const int lines_out = 32; //
    double y_result;
    double x_result;
    double y_out;
    double x_out;
    double max_result = -100;
    int i;
    int graph_driver, graph_mode, graph_error_code;
```

```

int max_x;
int max_y;
char symbols_out[10];
char maximum_out[50];

graph_driver = DETECT;
graph_mode = VGA;
detectgraph(&graph_driver, &graph_mode);
initgraph(&graph_driver, &graph_mode, NULL);

max_x = getmaxx();
max_y = getmaxy();
setviewport(10, 10, max_x - 10, max_y - 10, 0);
setlinestyle(0, 0, 3);
line(lines_out, max_y, lines_out, 0);           // vert
line(lines_out, max_y / 2, max_x, max_y / 2);   // gorizont
// Разметка по оси X
for (i = 1; i <= end_out; i++) { // Начинаем с 1, так как 0 - это 0п
    sprintf(symbols_out, "%dPI", i);
    outtextxy(lines_out + hatch_out * i, max_y / 2 + ten_num,
symbols_out);
    line(lines_out + hatch_out * i, max_y / 2 + ten_num / 2,
lines_out + hatch_out * i, max_y / 2 - ten_num / 2);
}
// Разметка по оси Y
for (i = -number_out; i <= number_out; i++) {
    sprintf(symbols_out, "%d", i);
    outtextxy(0, max_y / 2 - hatch_out * i / 2 - ten_num / 2,
symbols_out);
    line(lines_out + ten_num / 2, max_y / 2 - hatch_out * i / 2,
lines_out - ten_num / 2, max_y / 2 - hatch_out * i / 2);
}
setlinestyle(3, 0, 3);
line(lines_out + hatch_out * start_out, max_y,
lines_out + hatch_out * start_out, 0);

setlinestyle(0, 0, 3);
setviewport(0, 0, max_x, max_y, 0);
// graph
for (x_result = start_out * pi; x_result <= end_out * pi; x_result +=
0.001) {
    y_result = pow(sin(x_result), 3) + pow(cos(x_result), 3);
    x_out = x_result * x_approach;
    y_out = y_result * y_approach;
    if (y_result > max_result) {
        max_result = y_result;
    }
    setcolor(4);
    circle(lines_out + x_out, max_y / 2 - y_out, 1);
}

setlinestyle(3, 0, 3);

```

```

line(lines_out + hatch_out * end_out + 10, max_y,
      lines_out + hatch_out * end_out + 10, 0);

sprintf(maximum_out, " ", max_result);
settextstyle(0, 0, 2);
outtextxy(max_x / 3, max_y - lines_out, maximum_out);
getch();
closegraph();
return 0;
}

```

Структурная схема аппаратных средств.

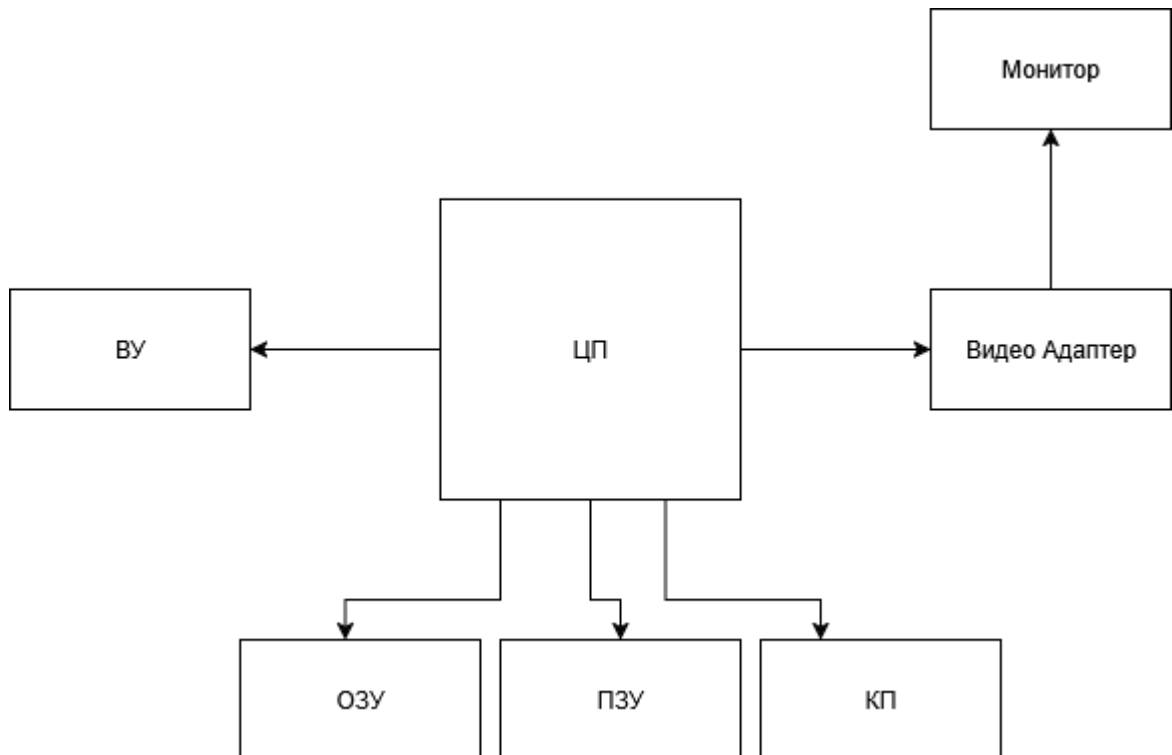
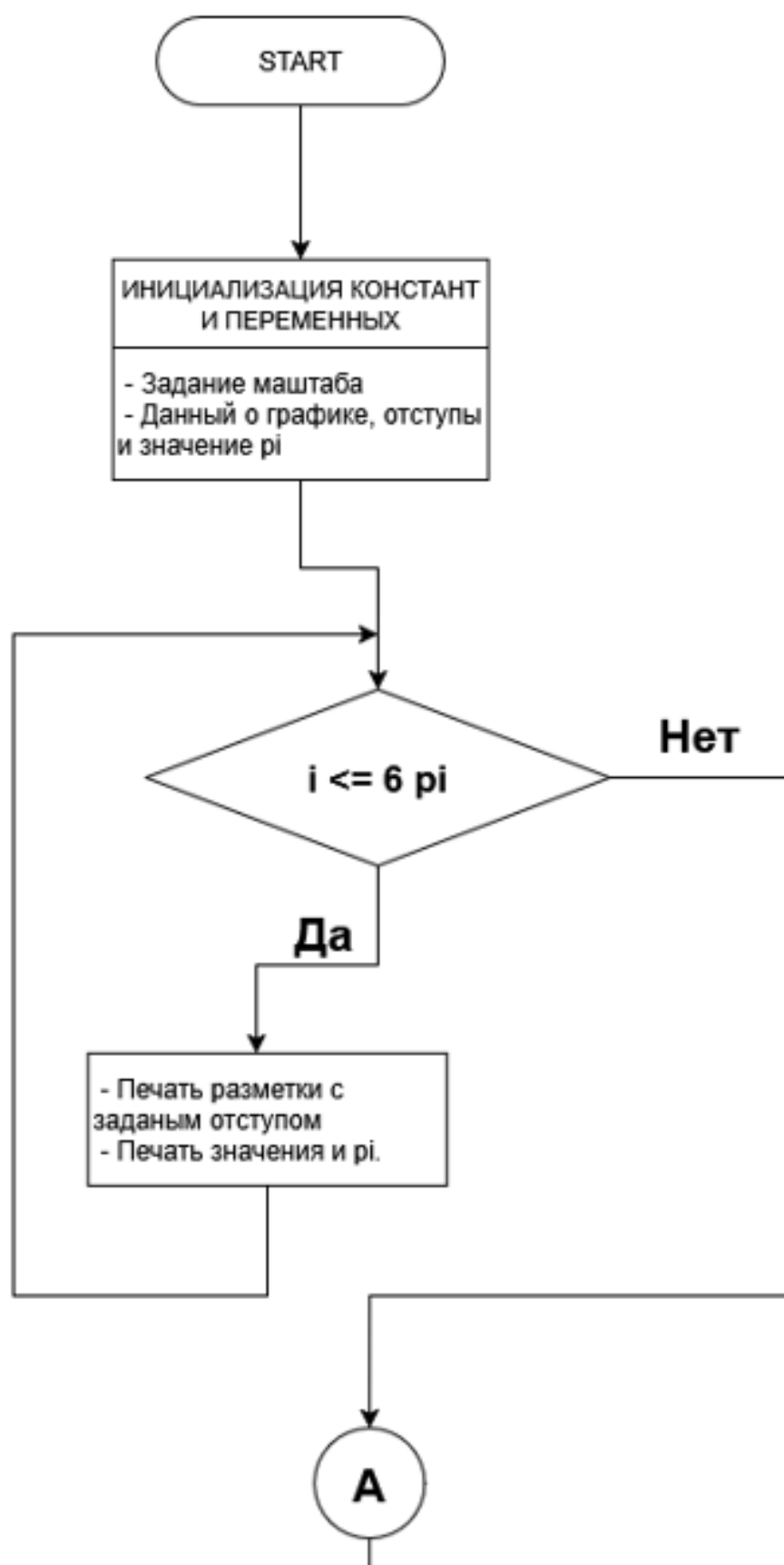


Схема алгоритма





Вывод

В результате выполнения работы были изучены принципы работы видеосистемы в графическом режиме и получены навыки по работе с ней.